

Střední průmyslová škola, Ústí nad Labem, Resslova 5



Little Mind

Dokumentace k ročníkové práci

Autor: Václav Suchý

Třída: 3ITA

Vedoucí práce: Bc. Vratislav Medřický

2024/2025

Prohlášení

Prohlašuji, že jsem ročníkovou práci na téma „Little Mind“ vypracoval samostatně a s použitím uvedené literatury a pramenů.

V Sedleci dne 13.06.2025

.....

Poděkování

Děkuji Bc. Vratislavu Medřickému za vedení mé práce a navrhnutí tohoto tématu

Anotace

Tato dokumentace obsahuje informace o vývoji hry Little Mind. Úvod představuje projekt a jeho cíle. V rešerši je vylíčená inspirace pro tento projekt. Technologie představují využité technologie a na co se konkrétně využili. Praktická část obsahuje realizaci projektu a závěr pojednává o úspěších a sebehodnocení.

Klíčová slova

Unity, hra, fobie, puzzle, choroba, příznaky, C#, interakce, úzkost

Obsah

Úvod	6
1 Teoretická část	7
1.1 Rešerše	7
1.1.1 Scarlet Deer Inn Demo	7
1.1.2 Supraland	8
1.2 Technologie	9
1.2.1 C#	9
1.2.2 Unity	9
1.2.3 Unity Asset – DOTween	9
1.2.4 Audacity	9
1.2.5 Visual Studio	9
1.2.6 JetBrains Rider	9
1.2.7 Claude AI	9
2 Praktická část	10
2.1 Návrhy	10
2.1.1 Návrh hráče	10
2.1.2 Návrh fobii	10
2.1.3 Návrh symptomů	10
2.2 Produktizace	11
2.2.1 Implementace Fobie	11
2.2.2 Implementace Symptomů	12
2.2.3 Systém interakcí	13
2.3 Popis pro uživatele	14
Závěr	15
Použitá literatura	16
Bibliografie	16
Seznam obrázků	17
Obsah média	18

Úvod

Cílem této ročníkové práce je vytvoření logiky hry s pomocí herního enginu Unity. Hra se bude zaměřovat na Alzheimerovu chorobu a fobie, a to zejména na arachnofobii, akrofobii, hematofobii a nyktofobii.

Hra „Little Mind“ bude zaměřena na hlubší pochopení těchto psychologických problémů prostřednictvím interaktivního zážitku. Projekt bude kombinovat různé herní mechaniky s psychologickými aspekty, což umožní hráčům získat zážitek z pohledu osoby trpící těmito nemocemi.

Projekt si klade za cíl vytvořit pět specifických herních úrovní, z nichž každá se bude věnovat jinému psychologickému aspektu. První úroveň se zaměří na arachnofobii prostřednictvím implementace pavouků a pavučin, druhá na akrofobii s využitím výškových prvků a žebříků. Další úrovně budou postupně představovat hematofobii, nyktofobii a projevy Alzheimerovy choroby.

1 Teoretická část

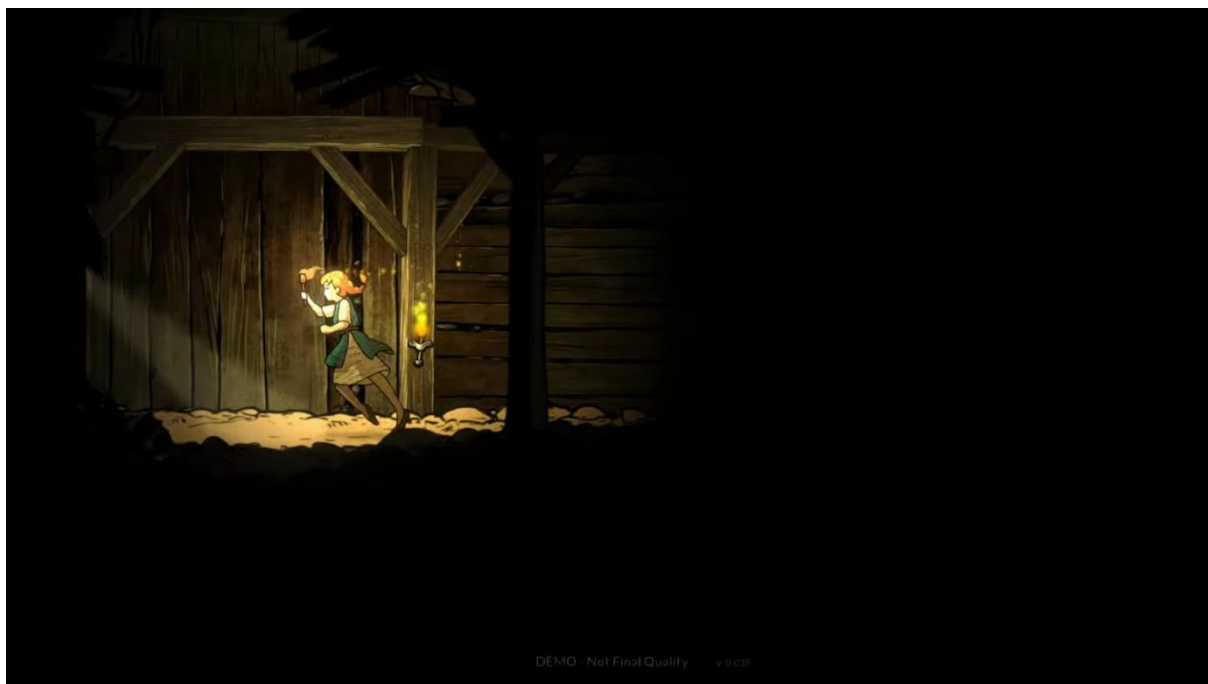
1.1 Rešerše

1.1.1 Scarlet Deer Inn Demo



Obrázek 1 - Scarlet Deer Inn vyšívání

Scarlet Deer Inn představuje unikátní plošinovou hru, která je v současnosti dostupná pouze ve formě dema, přičemž plná verze je stále ve vývoji. Hra vyniká svým osobitým výtvarným zpracováním, které kombinuje ručně vyšívané postavy s tradičně animovaným pozadím



Obrázek 2 - Scarlet Deer Inn převzatá mechanika

Klíčovým inspiračním prvkem inspirace se stala závěrečná herní mechanika z dema Scarlet Deer Inn, která pracuje s konceptem přežití prostřednictvím manipulace se světelnými zdroji. Protagonista nese pochodně, kterou využívá k zapalování dalších pochodní umístěných na stěnách prostředí. Setrvání ve tmě po určitou dobu vede k fatálním následkům způsobeným nespecifikovanou entitou.

Tuto mechaniku jsem adaptoval v úrovni, kde hráč ovládá postavu trpící nyktofobií. Zachoval jsem základní princip nutnosti udržování světelných zdrojů, avšak s významnou modifikací důsledků. Namísto smrti jsem implementoval mechanismus ztráty kontroly nad postavou, která při nadměrné expozici tmě automaticky vyhledává nejbližší dostupný zdroj světla.

Tato úprava lépe reflektuje psychologický aspekt nyktofobie a současně vytváří jedinečnou herní dynamiku bez nutnosti využití prvků mortality.

1.1.2 Supraland



Obrázek 3- Supraland

Supraland je inovativní first-person metroidvania vytvořená německým vývojářem Davidem Münnichem. Hra jedinečně kombinuje prvky ze série Zelda, Metroid a Portal, přičemž vytváří vlastní osobitý herní zážitek.

Hra Supraland slouží jako významný inspirační zdroj v oblasti designu logických hádanek.

1.2 Technologie

1.2.1 C#

Jazyk C# je vysokoúrovňový multiplatformní objektově orientovaný programovací jazyk pro platformu .NET.

V tomto programovacím jazyce budu programovat herní logiku hry. (Helsberg, et al., 2006)

1.2.2 Unity

Multiplatformní herní engine. Používá se jak pro vývoj 2D, tak i 3D her.

V tomto herním engine se bude dávat celá hra do kupy. (Haas)

1.2.3 Unity Asset – DOTween

Umožňuje jednoduché animace skrze API s krátkými příkazy, nabízí bohaté možnosti kontroly, callbacky, looping, easing, cesty, UI animace apod.

Tento plugin bude využit pro jednoduché animace UI. (Demigiant)

1.2.4 Audacity

¹FOSS, který umožňuje nahrávat a upravovat audio. Nabízí komplexní sadu pro zpracování zvuku. Mezi hlavní funkce tohoto programu se člení střih audia, míchání stop, změny rychlosti a odstranění šumu.

Program využiji pro úpravu zvukových efektů jako je: tlukot srdce a dýchání. (Audacity)

1.2.5 Visual Studio

Visual Studio je integrované vývojové prostředí (IDE) od společnosti Microsoft. Toto prostředí poskytuje komplexní sadu nástrojů pro vývoj softwaru, včetně pokročilého editoru kódu, debuggeru a podpory verzovacího systému. Visual Studio nabízí rozsáhlou integraci s Unity prostřednictvím specializovaných nástrojů a rozšíření.

V počátečních fázích projektu bylo využíváno jako primární prostředí pro práci s kódem. Později bylo nahrazeno JetBrains Rider z důvodu jeho pokročilejších funkcí pro vývoj s Unity. (Microsoft)

1.2.6 JetBrains Rider

JetBrains Rider je moderní crossplatformní vývojové prostředí specializované na .NET vývoj. Toto IDE kombinuje výkonný editor kódu s pokročilými nástroji pro refaktoring a analýzu kódu.

V projektu později využíván jako hlavní vývojové prostředí. (Jetbrains s.r.o.)

1.2.7 Claude AI

Claude AI je generativní umělá inteligence vyvinutá společností Anthropic, která se specializuje na zpracování přirozeného jazyka a asistenci při vývoji software. V rámci projektu je Claude AI využíván především pro Generování základních struktur kódu a konzultaci při řešení složitějších programátorských výzev (Anthropic PBC)

2 Praktická část

2.1 Návrhy

2.1.1 Návrh hráče

Jako ve většině FPS her by měl hráč být shopen ovládat hru WASD a pohybovat kamerou pomocí myši. Také by měl být schopen interagovat s prostředím.

2.1.2 Návrh fobii

Začal jsem návrhem jádra všech fobií, došlo mi, že každá fobie by měla mít aktuální intensitu a měla by být ovlivňována jednotlivými symptomy.

2.1.3 Návrh symptomů

Došlo mi, že jádrem všech symptomů je, že by měly být ovlivňovány fobií, takže by daná fobie by měl být schopna daný symptom zastavit či spustit a měla by také reagovat na intensitu dané fobie

2.2 Produktizace

2.2.1 Implementace Fobie

```
public abstract class AnxietyManager : MonoBehaviour
{
    [Header("Anxiety Settings")]
    [SerializeField] protected AnxietyManagerData Data;

    #region Protected props
    protected AnxietySymptom[] Symptoms;
    protected readonly HashSet<IAnxietySource> activeAnxietySources = new();
    [CanBeNull] protected Coroutine FadeCoroutine;
    #endregion
    #region Private props
    private float currentAnxiety = 0f;
    #endregion
    public virtual void RegisterAnxietySource(IAnxietySource anxietySource)
    {
        if (activeAnxietySources.Count == 0)
        {
            StartAllSymptoms();
        }

        if (FadeCoroutine != null)
        {
            StopCoroutine(FadeCoroutine);
        }
        activeAnxietySources.Add(anxietySource);
        IncreaseAnxiety(anxietySource.InitialAnxietyAmount);
        OnAnxietySourcesChanged();
    }

    public virtual void UnRegisterAnxietySource(IAnxietySource anxietySource)
    {
        if (activeAnxietySources.Remove(anxietySource))
        {
            if (activeAnxietySources.Count == 0)
            {
                StartFadeAnxiety();
            }
            OnAnxietySourcesChanged();
        }
    }

    protected float CurrentAnxiety
    {
        get => currentAnxiety;
        set
        {
            var newValue = Mathf.Clamp(value, 0f, Data.MaxAnxietyLevel);
            if (Mathf.Approximately(newValue, currentAnxiety)) return;

            currentAnxiety = newValue;
            OnAnxietyValueChanged();
        }
    }

    ...
}
```

Třída `AnxietyManager` má za úkol spravovat její symptomy a být ovlivňována zdroji úzkosti, Daná úzkost se zaregistruje do Hashovací tabulky dané fobie ve chvíli, kdy by měla působit na hráče. Třída každý snímek kontroluje zda je něco v Hashovací tabulce, pokud ano tak začne aktivovat symptomy a zvyšovat intenzitu úzkosti na základě jejího obsahu.

2.2.2 Implementace Symptomů

```
using UnityEngine;
```

```
public abstract class AnxietySymptom : MonoBehaviour
{
    protected float Intensity { get; set; } = 0f;

    /// <summary>
    /// Implement this method to define how the symptom activates.
    /// </summary>
    public abstract void ActivateSymptom(float intensity);

    /// <summary>
    /// Stop the symptom (e.g. end any coroutines or animations).
    /// </summary>
    public abstract void StopSymptom();

    /// <summary>
    /// Optionally override to define cleanup behavior.
    /// </summary>
    protected virtual void DeactivateSymptom()
    {
        enabled = false;
        Intensity = 0f;
        StopSymptom();
    }

    /// <summary>
    /// Called when the anxiety level changes.
    /// </summary>
    public virtual void OnAnxietyChanged(float newAnxiety)
    {
        Intensity = newAnxiety;
    }
}
```

Třída `AnxietySymptom` představuje abstraktní základní třídu pro implementaci různých symptomů úzkosti v herním prostředí Unity. Tato třída slouží jako základ pro vytváření specifických projevů úzkostných stavů, které mohou být dynamicky aktivovány a deaktivovány na základě aktuální úrovně úzkosti hráče.

2.2.3 Systém interakcí

```
using System;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.Serialization;

[Serializable]
public class Interaction
{
    public string ActionName;

    [FormerlySerializedAs("Action")]
    public InputActionReference ActionRef;
}

public interface IInteractable : IInteractionGroup
{
    public bool Interact(IInteractor interactor, InputAction invokedAction);
    public bool ToggleOutline(bool value);

    public event Action InteractionsChanged;
}

[Serializable]
public class GlobalInteractionGroup : IInteractionGroup
{
    [field: SerializeField]
    public string InteractGroupLabel { get; private set; }

    [field: SerializeField]
    public Interaction[] CurrentInteractions { get; private set; }
}

public interface IInteractionGroup
{
    public string InteractGroupLabel { get; }
    public Interaction[] CurrentInteractions { get; }
}

public interface IInteractor
{
    public Transform PickupPoint { get; }
    public void Pickup(IInteractable itemToPickUp);
    public IInteractable InteractableHolding { get; }

    public void SetHandTarget(HandTargetType targetType);
}

public enum HandTargetType
{
    none,
    key,
    twoHand,
    general,
}
```

Na jedné straně jsou definována rozhraní `IInteractable` a `IInteractions`, která určují základní vlastnosti a metody potřebné pro interakce zobrazení interakce v UI, jako například seznam dostupných akcí

(CurrentInteractions) nebo skupinové označení interakcí (InteractGroupLabel). Na druhé straně je třída GlobalInteractions, která slouží jako globální kontejner pro interakce hráče mimo specifické objekty (např. pohyb hráče, běh a krčení)

2.3 Popis pro uživatele

Hra vás uvítá hlavním menu, kde je možné si nastavit hlasitost a video nastavení, jako rozlišení, vertikální synchronizaci režim okna... Dále hráč může založit novou hru a začít hrát.

Hráč ovládá postavu pomocí WASD a myši, interaguje stiskem E a stiskem ESC zobrazí Pause menu

Závěr

V první polovině projektu jsem si dal za cíl vytvořit herní logiku pro ovládání hráče. Následně herní logiky všech fóbí a Alzheimerovi choroby, se kterými jsem spokojený. Myslím si že jsem se dost naučil a zapojil značné množství kreativity. Jsem také spokojen zejména se systémem interakcí. Dále jsem měl za úkol použít model hráče a jeho animace i s tímto jsem spokojený.

V druhé polovině projektu jsem měl za úkol implementovat narativní strukturu všech levelů pro každou fobii, jenž jsem nebyl schopen dokončit, neuvědomil jsem si jak moc je náročné vymýšlet prostředí. Čas vyhrazený pro implementaci systému ukládání jsem namísto věnoval tomuto obtížnému úkolu.

Měl jsem také za úkol propojit UI s logikou, jako je video nastavení. S touto částí jsem spokojen.

Použitá literatura

Bibliografie

Anthropic PBC. [Online] [Citace: 13. 06 2025.] <https://www.anthropic.com/claude>.

Audacity Team. About Audacity. [Online] Audacity Team. [Citace: 13. 06 2025.] <https://www.audacityteam.org/faq/>.

Demigiant. [Online] [Citace: 13. 06 2025.] <https://assetstore.unity.com/packages/tools/animation/dotween-hotween-v2-27676?srltid=AfmBOoquDCZ775t3oLsSyQHg5ly9gNMwG2kif012cL57g7gaJEF9oqYF>.

Haas, John. [Online] [Citace: 13. 06 2025.] http://www.daelab.cn/wp-content/uploads/2023/09/A_History_of_the_Unity_Game_Engine.pdf.

Helsberg, Anders, Wiltamuth, Scott a Golde, Peter. 2006. [Online] 2006. [Citace: 13. 06 2025.] https://books.google.cz/books?hl=cs&lr=&id=6L1Rm031qCkC&oi=fnd&pg=PA3&dq=C%23&ots=5yK3czTYyF&sig=abqgg4GtxNRfE5pBw0Tp3v2Kfvs&redir_esc=y#v=onepage&q=C%23&f=true.

Jetbrains s.r.o. [Online] [Citace: 13. 06 2025.] <https://www.jetbrains.com/rider/>.

Microsoft. [Online] [Citace: 13. 06 2025.] <https://visualstudio.microsoft.com/cs/>.

Seznam obrázků

Obrázek 1 - Scarlet Deer Inn vyšívání	7
Obrázek 2 - Scarlet Deer Inn převzatá mechanika	7
Obrázek 3- Supraland	8

Obsah média

1. Složka Build
 - a. Obsahuje spustitelný .exe soubor
2. Složka Docs
 - a. Obsahuje upravovatelný .docx soubor této dokumentace
 - b. Obsahuje .pdf soubor této dokumentace
 - c. Obsahuje .pptx soubor prezentace tohoto projektu
3. Složka Src
 - a. git repozitář unity projektu